Crypto

Digests, message authentication & random bytes.

Install & load

```
INSTALL crypto FROM community;
LOAD crypto;
```

01 At a glance

Compute cryptographic digests and HMACs directly from SQL values. Use secure random bytes for tokens and salts, or aggregate values in a specified order for reproducible checksums.

SQL values → Hash / HMAC → BLOB or hex

02 Core SQL surface

`crypto_hash(algorithm, value)`

Return a digest as BLOB; algorithm is a string.

`crypto_hmac(algorithm, key, message)`

Authenticate a message with a secret key; returns BLOB.

`crypto_random_bytes(length)`

Generate the requested number of cryptographically secure random bytes.

`crypto_hash_agg(algorithm, value ORDER BY ...)`

Aggregate values into a digest with explicit input order.

`to_hex(blob) / lower(text)`

Format binary digests as printable lowercase hexadecimal.

03 Keep in mind

Choose the algorithm explicitly

Common names include blake3, blake2b-512, sha2-256, sha2-512 and sha3-256. MD5 and SHA-1 are legacy choices, unsuitable for collision-resistant security.

04 Hash a string

```
SELECT lower(to_hex(
  crypto_hash('sha2-256', 'hello')
)) AS digest;
```

SHA-256 returns 32 bytes, rendered here as 64 hex characters.

05 Authenticate a message

```
SELECT lower(to_hex(crypto_hmac(
  'sha2-256', 'example-key', 'message'
))) AS signature;
```

Argument order is algorithm, key, message. Replace the example key.

06 Bytes & ordered aggregates

```
SELECT octet_length(crypto_random_bytes(32));

SELECT to_hex(crypto_hash_agg(
  'sha2-256', s ORDER BY s
)) AS digest
FROM (VALUES ('A'), ('B')) AS t(s);
```

ORDER BY belongs inside the aggregate call for reproducible ordering.

07 In practice

A checksum needs a data contract

Specify row order, input types and serialization. A digest of one field is not automatically an unambiguous digest of a whole row. Fast hashes are a separate hashfuncs extension.

Full docs & examples ↗ Source ↗